Sequential Causal Training of Windowed PINNs for The Lorenz ODE System

Ikramul Hasan Moral
Computer Science And Engineering
imoral223489@bscse.uiu.ac.bd
0112230489

Md. Abu Bakar
Computer Science And Engineering
mbakar223200@bscse.uiu.ac.bd
0112230200

Samiur Rahman Omlan
Computer Science And Engineering
somlan223195@bscse.uiu.ac.bd
0112230195

Md. Touhidul Islam
Computer Science And Engineering
mislam223435@bscse.uiu.ac.bd
0112230435

Fariha Islam
Computer Science And Engineering
fislam223431@bscse.uiu.ac.bd
0112230431

Dr. Muhammad Nomani Kabir
Professor, Dept of CSE
United International University
kabir@cse.uiu.ac.bd

***Abstract*—We study a physics-informed neural network (PINN) approximation of a single full-cycle orbit of the Lorenz three-state system on $t \in [0, 13.26446]$. We trained a single network on the full interval that produce a combined state root-mean-square error (RMSE) of 2.237. To adress this long horizon problem, we divide the interval in total 27 windows. Each window has a separate network and an initial state that is fixed to the predicted endpoint of the previous window. In each window, Adam reduces the residual in four stages. In each stage, we use causal weights and causal weighted loss to reduce the loss of all earlier points. Then, L-BFGS minimizes the loss using the unweighted residual mean. A numerical reference is used to monitor the training but is not used for optimization or stopping. We reported, in a single-seed run the windowed model gives a combined RMSE of 6.97×10^{-5} on 13,265 evaluation times and a dense-grid residual mean square of 1.87×10^{-7}. The full-interval and windowed runs also vary in terms of number of parameters, sampling and optimizer schedule. In contrast, these are described with the word configurations, not isolation of the effect of causal weighting.**

***Index Terms*—physics-informed neural network, Lorenz system, causal training, time windows, ordinary differential equation, windowed PINN**

I. Introduction

The Lorenz-system model is a three-state, nonlinear ordinary differential equation (ODE) that was obtained from a reduced flow model [1]. It has an initial-value problem which provides a simple test of the convergence of a learned continuous-time function from a specified trajectory over a long interval. A reference trajectory is provided by numerical integration. A PINN is trained with the residual of the ODE instead of the solution samples [2]. To train a a long horizon problem using traditional PINN is not suggested by previous works [3]. It couples early and late times, but the state at a later time is determined by the previous trajectory. To overcome this ordering, Wang *et al.* suggested the use of time-dependent residual weights PINN training [3]. An initial condition can also be met by construction using a hard trial solution [4]. We use both concepts, we split the time period in equal 27 windows and a separate network for every windows. The initial state is fixed with the endpoint predicted in one window of the next. The parameters of a trained window is copied to the next trainable window. This paper provides an implementation and its saved results. An interval that completes a full-cycle loop for the chosen initial state is $[0, 13.26446]$. We report one implementation and its saved result on $[0, 13.26446]$ which is essentially an closed orbit. The actual training goal and window handoff is then given and compared to a saved single-network training of the same initial-value problem. The state error, ODE residual, causal weights residual are also checked. The comparison is descriptive because multiple configuration options are changing simultaneously.

II. Problem and Related Work

In this section we introduce the mathematical goal of our study and place our configuration within the framework of existing neural-network techniques. We first describe the exact equations and the initial value problem of the Lorenz trajectory that we want the model to learn. Then we explain the existing methods that we used and then we merged them.

A. Lorenz System Initial-Value Problem

Our main object is a three-state nonlinear ordinary differential equation (ODE) that we need to solve. The Lorenz system general form [5] is:

$$\begin{aligned} \frac{dx}{dt} &= kl\left(\frac{1}{k^2+l^2} - \frac{1}{k^2}\right) yz, \\ \frac{dy}{dt} &= kl\left(\frac{1}{l^2} - \frac{1}{k^2+l^2}\right) xz, \\ \frac{dz}{dt} &= \frac{kl}{2}\left(\frac{1}{k^2} - \frac{1}{l^2}\right) xy. \end{aligned} \tag{1}$$

Suppose the state vector is given by $u = (x, y, z)^{\mathsf{T}}$. So,substituting $k = 2$ and $l = 1$ gives exactly $(-0.1,\ 1.6,\ -0.75)$ coefficient. The system implemented is:

$$\dot{x} = -0.1\, yz, \qquad \dot{y} = 1.6\, xz, \qquad \dot{z} = -0.75\, xy, \tag{2}$$

For our experiment we used the coefficients $(a_x, a_y, a_z) = (-0.1, 1.6, -0.75)$. We track the system starting from an initial state [5] of $u(0) = (0.5, 0.75, 1.0)^{\mathsf{T}}$ until a final time of $T = 13.26446$. On this particular rounded endpoint, the reference trajectory nearly returns to its starting state. It is very similar but not exactly closed so we call this interval a nearly closed orbit and not an exact period.

B. Related Work

In this study we are making an experimental investigation of a series of well known concepts. The approach adopted is a PINN that embeds the physics residual into the training objective [2]. We use a hard trial-function design [4] to ensure our starting state precisely at the point we want to implement it . Learning long trajectories is a well-known challenge due to the ordering of the time axis so we take inspiration from the causal residual weighting technique [3]. Lastly segmenting the timeline is based on ideas of causal sweeping and temporal decomposition [6]. Our implementation is an investigation of the performance of the individual components in this particular closed orbit of lorenz system.

III. Windowed PINN Method and Implementation

This section explains the implementation of windowed PINN with causal weigths. This section also contains the model construction, physics enforcement, window segmentation, its handoff and the sequence in how the optimizer works for this particular case.

A. Model Construction and Time Routing

We used several modelling strategies to approximate the solution for Lorenz system trajectory over $[0, T]$, where $T = 13.26446$ is one closed loop for the chosen initial state. We first tried one network to learn the entire trajectory at once. As it failed by stucking in a platue so we tried to split the time interval in equal segments.

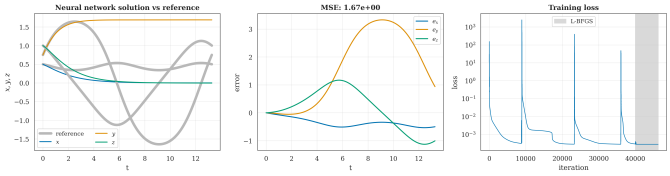


Fig. 1. Training result of a single network , entire time at once

We tried to keep the split window closer to 0.5, as small windows are better [3]. In first experiment, we used one network with 27 segmented time period to train the model sequentially. We noticed that, whenever the window is moved to the new one, the previous learning is lost or degraded. This results in significant increase in loss with RMSE of 2.237. To address these issues, we introduced WPINN (Windowed PINN). We split the total time $[0, T]$ equally into 27 windows and gave each window its own network. Each aprroximately 0.491 time units wide. Shorter window gives each network an easier piece to learn, therefore we tried to keep the window size closer to 0.5.

Each window has its own network $N_k(t; \theta_k)$ with 1 input layer (t), 4 hidden layers of 60 neurons with tanh unit, and with an output layer with 3 output (x, y, z). For the model parameter initialization we used xavier-uniform weights and zero biases. The 27 networks don't share parameters but the windows has shared boundary. The completed windows predicted endpoint for the boundary is used for the initial state of the next window.

B. Hard Initial State and Window Handoff

As Lorenz system is an initial value problem, we enforce the initial state of each window through a trial solution.

$$\hat{u}_k(t) = u_{0,k} + (t - t_k) N_k(t; \theta_k), \tag{3}$$

where $k = 0, \ldots, 26$, t_k is the starting time of window k, and $u_{0,k}$ is its fixed starting state. At $t = t_k$, the second term is zero, so $\hat{u}_k(t_k) = u_{0,k}$ regardless of the network parameters [4]. The first window uses the trial initial state, every later windows uses the predicted value of the last window as its initial state.

$$u_{0,k+1} = \hat{u}_k(t_{k+1}), \qquad k = 0, \ldots, 25. \tag{4}$$

Here window k+1 starts from a copy of the weights of window k then train on its own parameters. This makes the predicted values continuous at the window boundaries. As the starting point of the later windows is predicted, it can carry error from earlier windows.

C. Collocation, Residual, and Causal Loss

To predict and train the networks of each window, we split the time range into 1536 uniform collocation points per window. Ideally 1535 points for each window and 1 handoff point that becomes the initial state for the next window. While training, all the points in a single window are passed to the network at once which gives an output of shape $(1536, 3)$. Then we apply the trial solution to enforce the intial value. While calculating ODE residuals, pytorch automatic differentiation uses the vectors of x,y,z and create a graph that is differentiable. Pytorch autograd function gives the rate of change for all the collocation points with respect to time. And we use this value and the mathematical derivative to calculate the ODE residual.

$$r_k(t) = \frac{d\hat{u}_k(t)}{dt} - f\big(\hat{u}_k(t)\big). \tag{5}$$

For each ordered collocation time $t_{k,i}$, we take the mean of the three squared residual components:

$$L_{k,i} = \frac{1}{3}\left(r_{k,x}(t_{k,i})^2 + r_{k,y}(t_{k,i})^2 + r_{k,z}(t_{k,i})^2\right). \tag{6}$$

To solve a long horizon time dependent ODE system, only residual loss is not sufficient. Thats why use we used causal

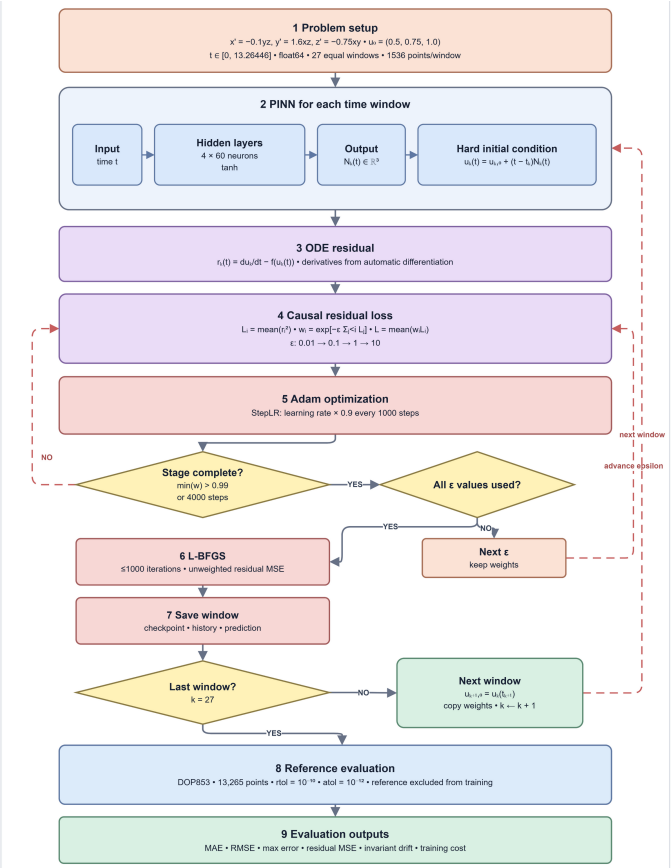


Fig. 2. Complete implementation pipeline

weights and causal weighted loss. The causal weight at a time depends on the accumulated residual loss at earlier times [3]:

$$w_{k,i} = \exp\left(-\epsilon \sum_{j<i} L_{k,j}\right), \tag{7}$$

$$\mathcal{L}_{k,\epsilon} = \frac{1}{n_k} \sum_{i=1}^{n_k} w_{k,i} L_{k,i}. \tag{8}$$

Here $n_k = 1536$. $L_{k,j}$ in (7) means the error at earliar points of i, where $j < i$ and $L_{k,i}$ means the loss at collocation point i in (8). $w_{k,i}$ denotes the causal weights of the collocation points. Here the causal weights are fixed while Adam optimizer calculated the gradient. We used four ϵ stages: 0.01, 0.1, 1, and 10 to reduce the earlier points error, and increase the weights. ϵ in the equation means the penalty for the models in each stages. Model must try to reduce the earlier loss to improve each points weights.

Only using ODE residual isn't enough for long horizon system like Lorenz system. And specially for the windowed algorithm, early times should be fitted before late times, as the errors accumulates and backtracking isn't possible. Therefore, later states depend on the earlier states, if earlier error is reduced, possibly the later error can be reduced as well. To address this, we first calculate the per-collocation point loss, $L_{k,i}$ which is the mean of the three squared ode residuals at time i. We assign weights for each point in time. Each points weight updates its value with the accumulated loss of all earlier points. The Causal loss (8) is the weighted mean of the residual loss. Weights are recomputed at every epoch. Later section explains the stopping rules of the stages and unweighted L-BFGS refinement steps.

D. Optimizer Sequence and Saved State

The causal loss defined in above, is minimized seperately in each window. Figure 2 summarize the sequence across the windows. Algorithm 1 gives the training steps. In each window a Adam optimizer and StepLR scheduler is created. The rate of the scheduler is 10^{-3} and it is multiplied by 0.9 for every 1,000 updates. It is maintained for the four stages of $\epsilon = (0.01, 0.1, 1, 10)$ across the window. There is a 4,000 update limit for each stage. When the saved minimum weight is above 0.99 or if there is an Adam update then the code will recalculate the causal weights at all collocation points at that particular window. If the minimum causal weight from the entire window among 1535 points is above 0.99 then it moves to the next ϵ stage. If it is below 0.99 and it reaches the epoch threshold then it moves to the next stage and records failure. When all four stages are completed then L-BFGS tries to minimize the loss using unweighted residual mean up to 1,000 epoch. To keep track of training we use a snapshot at every 50 epoch to keep training history.

The code saves the parameters of each window, transfers the endpoint state, and copies the parameters to the next trainable window. The config file has the same configuration as all modules were, initial states, training history, next window, PyTorch, NumPy etc. A different configuration in a config is rejected. The run also outputs stage status and residual snapshots. DOP853 values are captured at set up evaluative points. They do not enter the loss, gradient, or stopping test. After training, a dedicated custom function collects all the artifacts and evaluates the final piecewise model and its residual on the collocation and evaluation grids. Finally, a summary function computes error and invariant measures from these arrays. The reported full residual is one of the evaluation.

IV. Experimental Protocol

This section explain the experimental setup for the saved windowed run. It contains the configuration and reference information, the evaluation measures and the separate full interval comparison.

A. Baseline Reference

Table I shows the parameters used for the reported windowed run. We use SciPy's DOP853 [7] as the numerical reference, a relative tolerance of 10^{-10} and a absolute tolerance of 10^{-12}. It is sampled at 13,265 equally-spaced time points, and Distances away from training grid. This is a numerical reference at specified tolerances.

Algorithm 1 Sequential causal window training

Require: Ordered windows $W_0, \ldots, W_{26}$ and their uniform time points

Set $u_{0,0} \leftarrow (0.5, 0.75, 1.0)^\mathsf{T}$
for $k = 0, \ldots, 26$ **do**
  if $k > 0$ **then**
    Set $u_{0,k} \leftarrow \hat{u}_{k-1}(t_k)$
    Initialize $\theta_k \leftarrow \theta_{k-1}$
  end if
  Add t_{k+1} to the residual points if $k < 26$
  for $\epsilon \in (0.01, 0.1, 1, 10)$ **do**
    for at most 4,000 Adam steps **do**
      Network input: collocation time $t_{k,i} \in \mathbb{R}$,
      Here $i = 0, \ldots, 1535$
      Output: $N_k(t_{k,i}; \theta_k) = (x_{k,i}, y_{k,i}, z_{k,i}) \in \mathbb{R}^3$
      Compute trial solution: u_k by (3)
      Compute r by autodiff; sort times and L_i
      Set w_i by (7); detach w
      Backpropagate (8); Adam then StepLR
      if verified post-update min_$w_{k,i} > 0.99$ **then**
        break
      end if
    end for
    Record whether the threshold was met or capped
  end for
  Run L-BFGS on the unweighted residual loss
  Save window model, endpoint state, and config record
end for

TABLE I
WINDOWED-RUN CONFIGURATION

Setting	Value
Interval; windows	$[0, 13.26446]$; 27 equal windows
Network per window	1–60–60–60–60–3, tanh
Parameters	11,283 per window
Initial condition; precision	0.5, 0.75, 1.0; float64
Uniform collocation grid	41,446 unique points
Adam ϵ stages	$0.01, 0.1, 1, 10$
Stage threshold; epoch limit	min_$w_{k,i} > 0.99$; 4,000
Adam rate; scheduler	10^{-3}; $\times 0.9$ per 1,000 steps
L-BFGS budget	at most 1,000 iterations/window
Warm start; seed	Initialize $\theta_k \leftarrow \theta_{k-1}$; 0
Reference grid	13,265 uniform times

B. Measures and Comparison Run

On the reference grid, let define $e(t_j) = \hat{u}(t_j) - u_{\text{ref}}(t_j)$. The mean absolute error, RMSE, and maximum absolute error are reported. The combined RMSE is

$$\text{RMSE}_2 = \left(\frac{1}{n_{\text{eval}}} \sum_j \|e(t_j)\|_2^2 \right)^{1/2}, \tag{9}$$

while the combined maximum error is $\max_j \|e(t_j)\|_2$. The dense-grid residual mean square is the average of the three residual components squared, taken over the times of the evaluation. The comparison run is with the same network depth and width for the entire interval [0, T]. The windowed run simultaneously changes the number of networks, the grid, loss, warm start and the optimizer schedule. We measuere their final output and compare them.

TABLE II
WINDOWED-RUN ERROR AGAINST DOP853 ON 13,265 TIMES

State	MAE	RMSE	Maximum
x	1.89×10^{-5}	2.74×10^{-5}	1.13×10^{-4}
y	2.94×10^{-5}	4.96×10^{-5}	2.88×10^{-4}
z	2.78×10^{-5}	4.05×10^{-5}	1.30×10^{-4}
State norm	5.12×10^{-5}	6.97×10^{-5}	3.09×10^{-4}

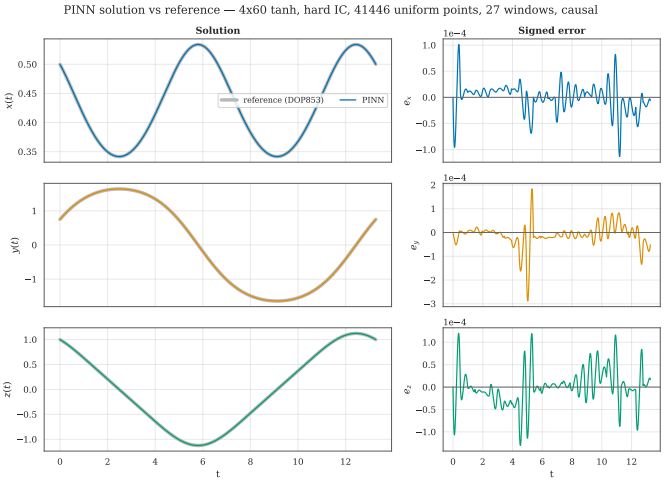


Fig. 3. PINN solution vs reference with signed errors.

V. RESULTS

This section reports the measurements taken from the completed training artifacts. We first look at what the windowed model does correctly and how well it follows the true path and physics. We then compare these results with the baseline reference and a single-network run for comparison with the behaviour of the model.

A. State Accuracy and Physics Checks

We compare the windowed PINN with an extremely accurate numerical reference which is SciPy's DOP853. As shown in Table II, our model achieved a combined RMSE of 6.97×10^{-5} across the 13,265 uniform times in our evaluation grid. The highest state error norm was 3.09×10^{-4}. Figure 3 plots these predictions against the reference over time. The trajectories are shown to overlap completely but the error show small peaks in the errors particularly at $t \simeq 5$.

We also measured how closely the model obeyed the underlying equations. The MSE of the ODE residual was very small on the evaluation grid at 1.872×10^{-7}. Also the residual MSE of windowed run on collocation grid is nearly same (1.874×10^{-7}). The DOP853 reference drifted less than 4×10^{-10} for perspective.

TABLE III
SAVED FULL-INTERVAL AND WINDOWED RUNS ON THE SAME IVP

Measure	One network	27 windows
Trainable parameters	11,283	304,641
Training points	39,793 LHS	41,446 uniform
Combined state RMSE	2.237	6.97×10^{-5}
Maximum state-error norm	3.346	3.09×10^{-4}
Collocation residual MSE	2.80×10^{-4}	1.87×10^{-7}
Recorded wall time (s)	4,529	881

Lastly, the hard trial solution ensures that the values of the state are exactly continuous at the inner 26 interfaces connecting the windows.

B. Comparison and Training Record

The differences with respect to the two saved configurations are shown in Table III. The RMSE value of the entire interval network is 2.237, which is much larger than the windowed run network's RMSE value of 6.97×10^{-5}. It is a notable result for the recorded settings of this initial value problem. All of the above are uncontrolled types of ablation in the concept of causal weighting [3], windowing [6] and warm starts as well as precision and even L-BFGS. In particular the model with the window is much more complicated with 27 times as many parameters and a different collocation grid. Reducing the parameter count of each network might result in similar error.

The 27 run windows had 34,765 Adam updates and 1000 L-BFGS closure evaluations per window. Closure evaluations are function calls made by L-BFGS, not iterations. Only two of the 108 causal stages ended at the 4,000-step cap without meeting the 0.99 weight threshold.

VI. DISCUSSION AND LIMITATIONS

There are a few limitations of our method. The trial solution [4] defines the initial state for each window exactly. The values remain continuous at the boundaries, but the derivatives can still change suddenly at the boundaries. This comparison with the single network is only a description of two saved runs. The two runs differed in a number of things, including the number of networks, grid, loss and schedule. There was not a windowed run that was saved without using causal weights, because [3] they are finding was respecting causality gives one or two order less error. This research applies to only one seed, one initial state and one coefficient set. Other initial states and other coefficient sets may result in different behavior. There is a weakness with the handoff. All windows that follow the first window begin from a predicted state and not the actual state. This means that errors from earlier windows get carried over to later windows. Only 2 of the 108 causal stages were at the 4,000-step limit but failed to reach the 0.99 weight limit. The causal weights Wang *et al.* vary depending on the number and spacing of the collocation points so if we increase the number of collocation points that will decrease the training loss. To obtain a fairer future test, one and only one training option will be changed at a time, with several seeds.

VII. CONCLUSION

Each of our 27 window PINN passes each of the tests on the nearly closed Lorenz-system interval $[0, 13.26446]$. The next window was predicted to be at the end. The saved run had a combined state RMSE of 6.97×10^{-5} in comparison to DOP853. A network designed separately to full intervals recorded an RMSE of 2.237. Future research will focus on matched configurations and initial states and generalise to problems with specified boundary conditions at each end.

REFERENCES

[1] E. N. Lorenz, "Maximum simplification of the dynamic equations," *Tellus*, vol. 12, no. 3, pp. 243–254, 1960.

[2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[3] S. Wang, S. Sankaran, and P. Perdikaris, "Respecting causality for training physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 421, p. 116813, 2024.

[4] I. E. Lagaris, A. Likas, and D. I. Fotiadis, "Artificial neural networks for solving ordinary and partial differential equations," *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.

[5] J. Matthews and A. Bihlo, "Pinnde: physics-informed neural networks for solving differential equations," *arXiv preprint arXiv:2408.10011*, 2024.

[6] M. Penwarden, A. D. Jagtap, S. Zhe, G. E. Karniadakis, and R. M. Kirby, "A unified scalable framework for causal sweeping strategies for physics-informed neural networks (PINNs) and their temporal decompositions," *Journal of Computational Physics*, vol. 493, p. 112464, 2023.

[7] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.